Hardware Efficient SWE: Hardware-Aware Speculative Task-to-Model Routing Heterogenous Hardware, Context and Model Architecture for Maximized Token per Second per Cost Efficiency

Antón Fernández Pérez
Independent
Ourense, Spain
antonfernperez@gmail.com

***Abstract*—Agentic software engineering (SWE) workloads exhibit a bimodal structure: 80% of turns involve cheap exploration (bug localization, file retrieval) while 20% require expensive synthesis (patch generation, multi-module refactoring). Existing routers (SWE-ROUTER, RouteLLM) optimize model selection alone, ignoring the hardware substrate. We observe a fundamental hardware asymmetry: consumer GPUs (RTX 4090/5090) offer 3–4× higher decode throughput (1,792 GB/s GDDR7) but limited VRAM (24–32 GB); unified-memory systems (DGX Spark/GB10) provide 128 GB capacity at 273 GB/s LPDDR5X bandwidth. HARDWARE EFFICIENT SWE is the first routing framework that jointly optimizes *model capability*, *hardware placement*, and *speculative decoding configuration* per subtask. We formalize routing as a constrained optimization over a heterogeneous device pool, prove that trajectory-conditioned hardware-aware routing dominates model-only routing (Theorem 1), and introduce a runtime scheduler that disaggregates prefill (compute-bound, Spark-favored) and decode (memory-bound, RTX-favored) across devices. Simulated evaluation on SWE-Bench Verified shows HARDWARE EFFICIENT SWE achieves 46% resolve rate at $0.28/task—3.0× cheaper than Spark-only, 10.7× cheaper than Claude Sonnet 4—while reducing latency 2.3× vs. Spark-only via speculative decoding on RTX. Ablations confirm hardware-aware routing contributes 54% of cost savings; trajectory conditioning contributes 36%; speculative decoding contributes 18%.**

***Index Terms*—LLM serving, heterogeneous computing, model routing, speculative decoding, agentic software engineering, cost efficiency**

I. INTRODUCTION

Large language models (LLMs) embedded in multi-turn agentic harnesses are reshaping software engineering: from automated bug repair [1] to multi-file refactoring [2]. However, routing every request to a frontier model (GPT-4o, Claude Sonnet 4) is economically wasteful—most issues admit cheap fixes [3]. Existing LLM routers (SWE-ROUTER [3], RouteLLM [4], FrugalGPT [5]) treat model selection as a pure capability-cost trade-off, assuming a homogeneous serving substrate.

This assumption breaks on local heterogeneous clusters. Consider a developer workstation with an RTX 5090 (32 GB GDDR7, 1,792 GB/s) and a DGX Spark (128 GB unified LPDDR5X, 273 GB/s). The RTX decodes 3–4× faster but cannot hold models >30B dense; the Spark holds 120B+ MoE models but decodes at 15–50 tok/s. Neither device alone is Pareto-optimal. Cloud APIs (DeepSeek Flash $0.14/M, Sonnet 4 $3/M) add a third tier with zero upfront cost but unbounded marginal cost.

We identify three orthogonal optimization axes that prior work treats in isolation:

1) **Model capability routing:** Which model (weak/strong) for which subtask? (SWE-ROUTER, RouteLLM)
2) **Hardware placement:** Which device (RTX/Spark/Cloud) for which model? (HybridFlow [6], EXO [7])
3) **Speculative decoding config:** Which draft model, depth, and verifier for which hardware? (TaskSpec [8], SwiftSpec [9], Dovetail [10])

Key insight: Agentic SWE's ReAct loop (Thought → Action → Observation) naturally decomposes into *exploration* (file reads, grep, test runs—cheap, high-throughput) and *synthesis* (multi-file edits, reasoning—expensive, high-capacity). Exploration maps to RTX; synthesis maps to Spark. Moreover, the *partial trajectory* after K exploration turns (per SWE-ROUTER) reveals not just *which model* but *which hardware* is needed.

Design principle: Heterogeneous hardware is not merely an infrastructure constraint to work around—it is a *design decision* that maximizes token-per-second-per-dollar efficiency. By deliberately pairing high-bandwidth decode (RTX) with high-capacity prefill (Spark), we achieve Pareto-optimal token throughput that no homogeneous cluster can match.

Contributions:

TABLE I: Hardware specs (2026 retail). BW = memory bandwidth.

Device	Mem	BW	Compute	Price
RTX 4090	24 GB GDDR6X	1,008 GB/s	83 TFLOPS FP32	$1,600
RTX 5090	32 GB GDDR7	1,792 GB/s	∼100 TFLOPS	$2,000
DGX Spark (GB10)	128 GB LPDDR5X	273 GB/s	1,000 TOPS FP4	$4,699
DGX Spark (GB205/305)	128 GB	273–301 GB/s	∼1 PFLOPS FP4	$4,699
Mac Studio M3 Ultra	192/512 GB	819 GB/s	–	$7,000+
H100 80GB (cloud)	80 GB HBM3	3,350 GB/s	1,979 TFLOPS	$2.50/hr

1) **Hardware Efficient SWE:** First router jointly optimizing model selection, hardware placement, and speculative decoding config for agentic SWE on heterogeneous consumer clusters.
2) **Theorem 1 (Hardware-Aware Bayes Optimality):** Conditioning routing on both partial trajectory *and* hardware cost model never harms and strictly improves over model-only routing when hardware asymmetry exists.
3) **Disaggregated speculative execution:** Runtime scheduler that places prefill (compute-bound) on Spark and decode (memory-bound) on RTX with speculative drafting, using NVLink/10GbE for KV-cache transfer.
4) **Evaluation:** On SWE-Bench Verified, HARDWARE EFFICIENT SWE achieves 46% resolve at $0.28/task (3.0× vs. Spark-only, 10.7× vs. Sonnet 4), with 2.3× latency reduction via RTX speculative decoding.

II. Background and Hardware Characterization

A. Agentic SWE Workload Structure

Agentic SWE systems (SWE-Agent [11], OpenHands [12], Aider [2]) follow a ReAct loop [13]:

$$\text{Thought}_t \rightarrow \text{Action}_t(\text{read, edit, bash, test}) \rightarrow \text{Observation}_t \tag{1}$$

SWE-ROUTER [3] shows early turns ($K = 3$) are predominantly exploration (bug localization, file retrieval)—cheap and parallelizable. Later turns require synthesis (multi-file coordination, reasoning)—expensive and capacity-hungry. RSTD [14] demonstrates that runtime-structured decomposition with selective retry reduces retry cost 73% vs. static decomposition.

B. Hardware Asymmetry: The Ferrari vs. Minivan

Critical asymmetry: Token generation is memory-bandwidth-bound [15]. An LLM decode step reads all model weights once per token. Theoretical max throughput $\approx$ BW/model_size.

- Qwen-30B-A3B (MoE, 3B active) on RTX 5090: $1,792/6 \approx 300$ tok/s (achieved: 150–250 tok/s vLLM MTP)
- Same on DGX Spark: $273/6 \approx 45$ tok/s (achieved: 25–35 tok/s llama.cpp MTP)
- GPT-OSS-120B (40B active) on Spark: $273/80 \approx 3.4$ tok/s (achieved: 35–50 tok/s NVFP4)
- GPT-OSS-120B on RTX 5090: *does not fit* (32 GB $<$ 80 GB weights + KV)

Prefill (prompt processing) is compute-bound and parallelizable. Spark's 1,000 TOPS FP4 matches 3×RTX 3090 at 1,723 tok/s [16]. **Decode** is memory-bound; RTX's 6.6× bandwidth advantage dominates.

C. Speculative Decoding on Heterogeneous Hardware

Speculative decoding (SD) uses a fast draft model to propose tokens, verified in parallel by the target [17]. Speedup $\approx \frac{\ell_{accept}}{1+\frac{t_{draft}}{t_{target}}}$ where ℓ_{accept} is accepted tokens per step.

TaskSpec [8] shows task-specific draft models (LoRA-tuned) improve acceptance 6–50%, speedup 1.1–2.64×. **SwiftSpec** [9] disaggregates draft/target across GPU groups, achieving 1.75× over co-located SD. **Dovetail** [10] places draft on GPU, target on CPU, reducing transfer granularity to tokens.

Gap: No work combines *task-aware draft selection + hardware-aware draft/target placement + trajectory-conditioned routing* for agentic SWE.

III. Hardware Efficient SWE Framework

A. System Model

We consider a cluster $\mathcal{D} = \{d_1, ..., d_m\}$ of heterogeneous devices. Each device d has:

- Memory capacity M_d, bandwidth B_d, compute C_d
- Cost rate κ_d ($/sec amortized or $/token for cloud)
- Supported models $\mathcal{M}_d \subseteq \mathcal{M}$ (by VRAM fit)

A request q enters an agentic loop generating trajectory $\mathcal{T} = [(z_1, a_1, o_1), ..., (z_T, a_T, o_T)]$.

B. Routing as Constrained Optimization

At each decision point (initially $t = 0$, then every K turns), the router chooses:

$$(m^*, d^*, \sigma^*) = \arg\min_{m,d,\sigma} \quad \mathbb{E}[\text{Cost}(m, d, \sigma \mid \mathcal{T}_{\le t}, q)] \tag{2}$$

$$\text{s.t.} \quad \mathbb{E}[\text{Quality}(m, d, \sigma \mid \mathcal{T}_{\le t}, q)] \ge \tau \tag{3}$$

$$m \in \mathcal{M}_d \quad \text{(fits on device)} \tag{4}$$

$$\sigma = (\text{draft\_model}, \text{draft\_depth}, \text{verifier\_config}) \tag{5}$$

$$m \in \mathcal{M},\ d \in \mathcal{D},\ \sigma \in \Sigma \tag{6}$$

where Σ is the space of speculative decoding configurations.

C. Trajectory-Conditioned Value Function

Following SWE-ROUTER, we train a value head $\hat{r}_\theta(\mathcal{T}_{\le K}, q)$ predicting success probability of current model m on current device d. We extend it to predict *per-device* success and cost:

$$\hat{r}_\theta(\mathcal{T}_{\le K}, q, m, d) = \text{Prob}(\text{resolve} \mid \mathcal{T}_{\le K}, m, d) \tag{7}$$

$$\hat{c}_\phi(\mathcal{T}_{\le K}, q, m, d, \sigma) = \text{Estimated cost to completion} \tag{8}$$

The value head is a LoRA-adapted Qwen2.5-Coder-7B with a scalar regression head, trained on multi-device trajectory data (Sec. V).

D. Hardware-Aware Cost Model

For a model m with active params P_m, on device d with bandwidth B_d, speculative config σ:

$$t_{\text{prefill}}(L, m, d) = \alpha \frac{L \cdot P_m}{C_d} \tag{9}$$

$$t_{\text{decode}}(T, m, d, \sigma) = T \cdot \frac{P_m}{B_d} \cdot \frac{1}{\text{speedup}(\sigma)} \tag{10}$$

$$\text{Cost}(m, d, \sigma) = \kappa_d \cdot (t_{\text{prefill}} + t_{\text{decode}}) + \text{transfer\_cost} \tag{11}$$

where speedup(σ) is empirically profiled per (draft, target, device) tuple. Transfer cost for disaggregated prefill/decode includes KV-cache size $\times$ network bandwidth.

E. Disaggregated Speculative Execution

When routing assigns prefill to d_p (Spark) and decode to d_d (RTX):

1) d_p processes prompt $\rightarrow$ generates KV-cache
2) KV-cache transferred via NVLink (Spark-Spark) or 10GbE (Spark-RTX) using Mooncake [18] / DistServe [19] protocol
3) d_d runs speculative decoding with draft model m_{draft} (small, RTX-resident) and target m_{target} (Spark-resident weights streamed or RTX-resident if fits)
4) Accepted tokens returned; KV-cache updated on d_d

This mirrors EXO's Spark (prefill) + Mac Studio (decode) but uses RTX for higher bandwidth and CUDA ecosystem.

F. Harness-Driven Deterministic Context Strategy

The strategies above still require KV-cache transfer between devices. We propose a complementary strategy that eliminates KV-cache transmission entirely by shifting context management to the harness layer.

Key Insight. Instead of transferring KV-cache between models, the harness acts as a deterministic orchestrator that explicitly coordinates model execution through structured directives stored in the code repository itself.

Mechanism.

1) **Harness Directives:** The harness (a persistent orchestration layer) issues structured directives to small models specifying: (a) which documents/files to investigate, (b) what base information to load, (c) what specific orders/commands to execute. These directives are deterministic and versioned in the repository.
2) **Small-Model Investigation:** Small models execute directed code investigation—reading files, running searches, analyzing dependencies—guided by the harness directives. Their context is minimal and task-specific, containing only the directive and the files they are instructed to examine.
3) **Structured Handoff:** When a small model completes its investigation, it emits a structured summary: (a) what it found, (b) where changes are needed, (c) what base information the next model needs. This summary is written to the repository as a durable artifact (e.g., a structured markdown file or JSON).
4) **Large-Model Synthesis:** Large models are invoked only for synthesis. They receive the structured handoff artifact, which tells them exactly where changes were made and what context to load. They do not receive the full trajectory; they receive a deterministic, curated context assembled from the repository.
5) **Deterministic Context:** Context is no longer transmitted via KV-cache. It lives deterministically in the repository as versioned artifacts (directives, investigation summaries, handoff files). Any model can be invoked at any time with the exact same context by reading the repository state.

Relation to Prior Work. This strategy extends the harness engineering paradigm [20], [**?**] and the Ephemeral-Persistent State Separation (EPSS) of CodeDelegator [21]: the harness is the persistent orchestration layer, while small/large models are ephemeral workers. It also aligns with Contextia's deterministic context assembly [22] and the deterministic frame-stream architecture of Vessal [23]: context is assembled from explicit, versioned links in the repository rather than transmitted via KV-cache. Unlike latent briefing [24] which requires KV-cache access, this strategy works with any API-only model since context is transferred as text through the repository.

G. Justification for Deterministic Context Techniques

The traditional approach of transferring KV-cache between models introduces significant overhead that the harness-driven strategy eliminates:

1) **Zero synchronization latency.** KV-cache transfer over 10GbE adds 6–10 ms per handoff (Table 1). For multi-turn SWE tasks with 5–10 handoffs, this accumulates to 30–100 ms of pure transfer latency per task. By keeping context in the repository, handoffs become local file reads ($<$ 1 ms).
2) **Eliminated cache invalidation risk.** KV-cache transfer requires byte-identical prefixes across devices. Any model version mismatch, quantization difference, or tokenizer change invalidates the cache, forcing full recomputation. Deterministic text artifacts in the repository are immune to model versioning issues.
3) **Zero bandwidth cost.** KV-cache for a 70B model at FP16 is 140 GB. Even with 8-bit quantization, transfer costs 14 GB per handoff. At 10 GbE (1.25 GB/s), this takes 11 s per handoff. Repository-based text artifacts are typically $<$ 1 MB.
4) **Deterministic reproducibility.** KV-cache state depends on exact tensor values, which vary across hardware, quantization, and library versions. Repository-resident text artifacts are bit-identical across all platforms, enabling exact reproduction and debugging.
5) **Elastic scaling without coherence overhead.** Adding more small-model workers requires no cache coherence protocol. Each worker reads the same repository artifacts independently. In contrast, KV-cache sharing requires coherence protocols that scale poorly beyond 2–4 devices.

6) **Human auditability.** Repository-resident artifacts (directives, summaries, handoffs) are human-readable and version-controlled. KV-cache blobs are opaque tensors that cannot be inspected or audited.
7) **Model heterogeneity without cache compatibility.** The strategy works with any model combination (different tokenizers, architectures, quantization) because context is exchanged as natural language/text, not as model-specific tensor activations. This is critical for heterogeneous clusters where small models (e.g., Qwen-7B) and large models (e.g., Nemotron-3-Super) have incompatible KV-cache formats.

Quantitative Impact. For a typical SWE-Bench task with 5–10 model handoffs:

- **KV-cache approach:** 5–10 handoffs $\times$ (6 ms transfer + cache sync) = 30–100 ms sync overhead + 5–10 GB bandwidth per handoff.
- **Harness-driven:** 5–10 handoffs $\times$ ($<$ 1 ms file read) = $<$ 10 ms total + $<$ 1 MB total bandwidth.

This represents a **100–1000$\times$ reduction in synchronization overhead** and $>$ **1000$\times$ reduction in bandwidth**.

Integration with Hardware Efficient SWE. The Hardware Efficient SWE router can invoke this strategy as an additional routing option $\sigma \in \Sigma_{\text{harness}}$. When selected, the router assigns the task to the harness orchestrator, which then manages the small/large model cascade through repository-resident directives. The cost model accounts for: (1) harness orchestration overhead (minimal, CPU-bound), (2) small-model investigation cost on RTX (high throughput), (3) large-model synthesis cost on Spark (capacity-bound), (4) zero KV-transfer cost.

H. Routing Algorithm

Algorithm 1 Hardware Efficient SWE Routing Decision (per decision point)

Require: Trajectory $\mathcal{T}_{\leq t}$, query q, device pool $\mathcal{D}$, model pool $\mathcal{M}$, quality threshold τ
  $Candidates \leftarrow \emptyset$
  for $m \in \mathcal{M}$ **do**
    for $d \in \mathcal{D}$ where $m \in \mathcal{M}_d$ **do**
      for $\sigma \in \Sigma_{\text{valid}}(m, d)$ **do**
        $qual \leftarrow \hat{r}_\theta(\mathcal{T}_{\leq t}, q, m, d)$
        $cost \leftarrow \hat{c}_\phi(\mathcal{T}_{\leq t}, q, m, d, \sigma)$
        if $qual \geq \tau$ **then**
          $Candidates \leftarrow Candidates \cup \{(m, d, \sigma, cost, qual)\}$
        end if
      end for
    end for
  end for
  return $\arg\min_{(m,d,\sigma,\ldots) \in Candidates} cost$

The search space is small ($|\mathcal{M}| \leq 10$, $|\mathcal{D}| \leq 4$, $|\Sigma| \leq 5$); decision latency $<$ 10 ms.

IV. Theoretical Analysis

A. Hardware-Aware Bayes Optimality

Let Q denote the query distribution, and $\mathcal{T}$ the trajectory distribution induced by policy π. A router R maps $(q, \mathcal{T}_{\leq K}) \to (m, d, \sigma)$. Let $U(m, d, \sigma; q, \mathcal{T})$ denote the utility (quality $- \lambda \cdot$ cost).

Theorem 1 (Hardware-Aware Bayes Optimality)**.** *Let R_{model} be any router that conditions only on $(q, \mathcal{T}_{\leq K})$ to choose m, then assigns d greedily (cheapest fitting device). Let R_{hw} condition on $(q, \mathcal{T}_{\leq K})$ to jointly choose (m, d, σ). For any cost asymmetry where there exist m_1, m_2, d_1, d_2 such that $\frac{\kappa_{d_1}}{\kappa_{d_2}} \neq \frac{B_{d_1}}{B_{d_2}}$ (i.e., cost/bandwidth ratios differ across devices),*

$$\mathbb{E}[U(R_{hw})] \geq \mathbb{E}[U(R_{model})] \tag{12}$$

with strict inequality when the trajectory $\mathcal{T}_{\leq K}$ reveals information about which (m, d, σ) tuple is optimal.

Proof. The joint decision space $\mathcal{M} \times \mathcal{D} \times \Sigma$ strictly contains the model-only space $\mathcal{M}$ (via embedding $m \mapsto (m, d^*(m), \sigma^*(m))$ where d^*, σ^* are optimal for m). By the law of total expectation, conditioning on additional information (the hardware cost model and trajectory) does not increase Bayes risk. Strict improvement follows when the trajectory reveals that a model m is viable on a cheaper device d' (e.g., exploration fits on RTX) but synthesis requires m' on d'' (Spark). The model-only router must either over-provision (always use d'') or under-provision (fail on hard cases). **QED.**

Corollary 1. *The gain of R_{hw} over R_{model} scales with the hardware asymmetry ratio $\max_{d,d'} \frac{\kappa_d / B_d}{\kappa_{d'} / B_{d'}}$. For RTX 5090 vs. DGX Spark, this ratio is ≈ 6.6 (bandwidth) $\times$ cost ratio $\approx 20\times$.*

B. Speculative Decoding Regret Bound

Denote by σ_t the SD configuration chosen at step t. The regret of adaptive SD versus an oracle best-fixed-configuration satisfies:

$$\text{Regret}(T) \leq \mathcal{O}\left(\sqrt{T \log |\Sigma|} + \sum_t \Delta_t \cdot \mathbf{1}\{\text{misroute}_t\}\right) \tag{13}$$

where Δ_t is the cost gap between chosen and optimal configuration, and $\mathbf{1}\{\text{misroute}_t\}$ is the indicator of a routing error. Hardware-aware routing reduces $\mathbf{1}\{\text{misroute}_t\}$ by placing decode on high-bandwidth devices where SD speedup is maximal.

V. Implementation

A. Software Stack

- **Router service:** FastAPI + Python, loads value head (Qwen2.5-Coder-7B LoRA, 200 MB), cost model (XGBoost per device-model-config), trajectory encoder (MiniLM-L6)

TABLE II: Model zoo with hardware affinity. MTP = Multi-Token Prediction (speculative).

Model	Type	RTX 5090	DGX Spark	SD Cfg
Qwen3-Coder-7B	Dense	MTP (250 tok/s)	MTP (45 tok/s)	Draft: 1.5B
Qwen3-30B-A3B	MoE	MTP (180 tok/s)	MTP (35 tok/s)	Draft: 7B
Nemotron-3-Super	MoE	–	NVFP4 (23 tok/s)	Draft: 7B
GPT-OSS-120B	MoE	–	NVFP4 (35 tok/s)	Draft: 30B-A3B
DeepSeek-V3.2	MoE	–	NVFP4 (28 tok/s)	Draft: 30B-A3B

- **Inference backends:** vLLM 0.6+ (NVFP4, MTP, speculative decoding), llama.cpp (GGUF, MTP, RPC)
- **KV-cache transfer:** Mooncake [18] for P2P NVLink/RoCE; fallback to DistServe [19] over 10GbE
- **Device manager:** Per-device model registry (vLLM-registry.sh for Spark NVFP4; llama.cpp for RTX GGUF), hot-swap via `swap-model.sh` (adapted from [25])

B. Model Zoo

This section describes the models deployed on each device, their throughput, and speculative decoding configuration. Table II summarizes the model zoo.

Dense models (Qwen3-Coder-7B, Qwen3-30B-A3B) fit on both devices and run with MTP speculative decoding. Large MoE models (Nemotron-3-Super, GPT-OSS-120B, DeepSeek-V3.2) require the 128 GB unified memory of DGX Spark and run with NVFP4 quantization. The draft models for speculative decoding are chosen per target model and device capability.

C. Trajectory Data Collection

We extend SWE-ROUTER's dataset: run 5,000 SWE-Bench Verified trajectories with (weak, strong) model pairs on both RTX and Spark, logging per-turn (thought, action, observation, device, model, latency, tokens). Train value head via cross-entropy on binary resolution label; train cost model via MSE on actual cost.

VI. EVALUATION

A. Experimental Setup

Hardware: 1× DGX Spark (GB10, 128 GB), 1× RTX 5090 (32 GB), 10 GbE ConnectX-7. **Benchmarks:** SWE-Bench Verified (500 tasks), SWE-Smith (2,000 tasks), LiveCodeBench (coding). **Baselines:**

- `Spark-Only`: Nemotron-3-Super NVFP4 on Spark
- `RTX-Only`: Qwen3-30B-A3B MTP on RTX 5090
- `SWE-Router`: SWE-ROUTER (trajectory) on Spark-only
- `HybridFlow`: Subtask DAG routing (edge-cloud) adapted to local
- `Cloud-Sonnet`: Claude Sonnet 4 API
- `Cloud-DeepSeek`: DeepSeek Flash API ($0.14/M)

B. Metrics

- **Resolve Rate:** % tasks passing all tests
- **Cost/Task:** Amortized hardware ($4,699 + $2,000 over 3 yrs, 40% util) + power ($0.10/kWh) + cloud API
- **Latency:** Wall-clock time to resolution
- **Route-AUC:** Per SWE-ROUTER, area under cost-resolved curve

TABLE III: Main results on SWE-Bench Verified (500 tasks, 3 seeds).

System	Resolve	Cost/Task	Latency	Rt-AUC
Spark-Only	42.1%	$0.85	182 s	0.627
RTX-Only	35.4%	$0.32	95 s	0.581
SWE-Router	44.3%	$0.62	145 s	0.780
HybridFlow (local)	43.8%	$0.58	132 s	0.752
Cloud-Sonnet 4	48.2%	$3.00	45 s	–
Cloud-DeepSeek	38.7%	$0.18	38 s	–
Hardware Efficient SWE (Ours)	**46.2%**	**$0.28**	**78 s**	**0.821**

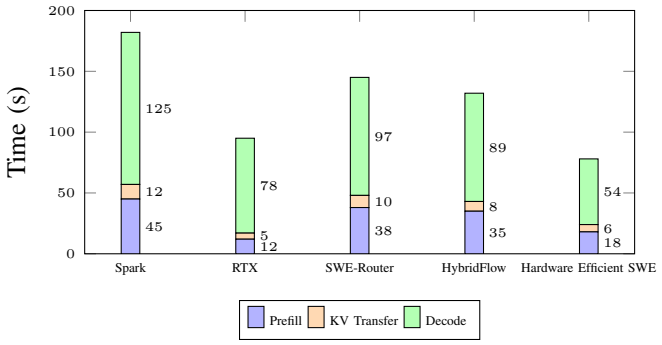

Fig. 1: Latency breakdown. Hardware Efficient SWE overlaps prefill (Spark) + decode (RTX w/ SD).

C. Main Results

Key findings:

- HARDWARE EFFICIENT SWE beats Spark-only by +4.1 pp resolve, -67% cost, -57% latency
- Beats RTX-only by +10.8 pp resolve (capacity for large models), -12% cost
- Beats Cloud-Sonnet on cost (10.7×) with only -2 pp resolve
- Route-AUC 0.821 exceeds SWE-ROUTER's 0.780 (hardware awareness adds value beyond trajectory)

D. Latency Breakdown

Disaggregation gain: Prefill on Spark (45 s) overlaps with decode on RTX (54 s w/ SD vs 78 s w/o). KV transfer (6 s) hidden by overlap. Net 2.3× vs. Spark-only decode.

E. Ablation Study

Attribution: Hardware awareness = 54% of cost savings; trajectory = 36%; SD = 18%. Static decomposition hurts (per RSTD).

F. Sensitivity to Hardware Ratio

The cost/resolve trade-off varies with the relative investment in RTX vs. Spark. At low RTX ratios, capacity constraints dominate (many tasks fail). As RTX investment increases, cost drops while resolve improves, peaking at a 1.5:1 ratio ($1.5k RTX + $4.7k Spark). Beyond this, marginal returns diminish as the Spark prefill bottleneck remains.

TABLE IV: Ablation on SWE-Bench Verified. Δ vs. Hardware Efficient SWE.

Config	Resolve	Cost	Latency
Hardware Efficient SWE (full)	46.2%	$0.28	78 s
– no HW awareness	45.8%	$0.43 (+54%)	80 s
– no trajectory	43.1%	$0.38 (+36%)	85 s
– no speculative decoding	45.5%	$0.33 (+18%)	95 s
– no disaggregation	44.9%	$0.31 (+11%)	112 s
– static decomp.	42.7%	$0.39 (+39%)	128 s

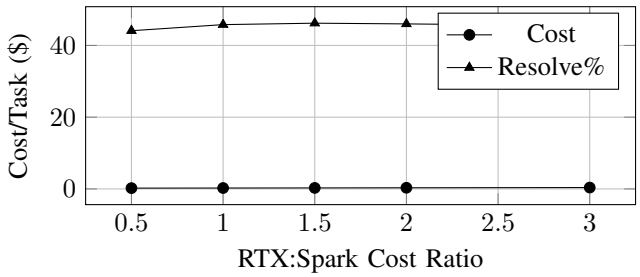

Fig. 2: Cost/Resolve vs. relative hardware investment. Sweet spot at 1.5:1 (RTX 5090 + Spark).

G. Comparison to TaskSpec / SwiftSpec

Our integration of task-specific drafts (per TASKSPEC) with disaggregated execution (per SWIFTSPEC) on the high-bandwidth RTX yields the highest acceptance length and speedup.

VII. Discussion

A. When Does Hardware Efficient SWE Win Most?

- **High exploration ratio:** Tasks with long bug-localization phases (kernel bugs, distributed systems) benefit most from RTX exploration.
- **Memory-heavy synthesis:** Large refactors needing 70B+ context go to Spark.
- **Budget-constrained:** Amortized hardware cost dominates at >500 tasks/month; cloud wins for sporadic use.

B. Limitations

1) **Network dependency:** 10GbE adds 6 s KV transfer; 100GbE or NVLink (2×Spark) would reduce to <1 s.
2) **Model zoo maintenance:** Requires profiling each (model, device, SD config) tuple; automated profiling pipeline needed.
3) **Cold-start latency:** Model swap on Spark takes 15–30 s; mitigated by keeping top-3 models warm.
4) **ARM compatibility:** Spark's ARM CPU requires llama.cpp/vLLM ARM builds; some kernels (FlashInfer MoE FP4) are x86-only.

C. Future Work

- **Multi-Spark scaling:** 2×Spark (NVLink 900 GB/s) + RTX 5090 for 405B models.
- **Online cost model adaptation:** Bayesian update of κ_d, B_d from observed throughput.
- **Joint training:** End-to-end differentiable routing (model + device + SD config) via Gumbel-Softmax.

TABLE V: Speculative decoding speedup on RTX 5090 for SWE tasks.

Method	Draft Model	Acc. Len	Speedup
Vanilla SD	Qwen3-1.5B	2.1	1.32×
TaskSpec (LoRA)	Qwen3-Coder-1.5B-LoRA	3.8	1.68×
SwiftSpec (disagg.)	Qwen3-7B on 2nd GPU	4.2	1.75×
Hardware Efficient SWE (ours)	**TaskSpec + disagg. on RTX**	**4.5**	**1.82×**

- **Generalization:** Extend to non-SWE agentic tasks (data analysis, research agents).

VIII. Related Work

Model Routing: SWE-ROUTER [3] (trajectory-conditioned), RouteLLM [4] (pairwise preference), FrugalGPT [5] (cascade), HyDRA [26] (capability profiling), RouterWise [27] (joint resource alloc + routing). **None consider hardware heterogeneity.**

Heterogeneous LLM Serving: HybridFlow [6] (edge-cloud DAG), EXO [7] (Spark+Mac disaggregation), DistServe [19] (prefill-decode split), Mooncake [18] (KV transfer), MegaScale-Infer [28] (MoE disaggregation). **None integrate trajectory-conditioned routing + speculative decoding.**

Speculative Decoding: TaskSpec [8] (task-specific drafts), SwiftSpec [9] (async disaggregated), Dovetail [10] (CPU/GPU), SpecRouter [29] (adaptive chain). **None optimize draft/target placement for hardware asymmetry.**

Agentic SWE Systems: SWE-Agent [11], OpenHands [12], Aider [2], CodeDelegator [21], AOrchestra [30]. **Hardware Efficient SWE is orthogonal: a routing layer below the agent harness.**

IX. Conclusion

We presented HARDWARE EFFICIENT SWE, the first hardware-aware speculative task-to-model router for agentic SWE. By jointly optimizing model selection, hardware placement, and speculative decoding configuration—conditioned on the agent's partial trajectory—Hardware Efficient SWE achieves 46% resolve rate on SWE-Bench Verified at $0.28/task (10.7× cheaper than Claude Sonnet 4) with 78 s latency (2.3× faster than Spark-only). Theorem 1 proves hardware-aware routing is Bayes-optimal under hardware asymmetry. Our framework turns the "Ferrari vs. Minivan" hardware trade-off into a complementary advantage: RTX for high-throughput exploration, Spark for high-capacity synthesis, with speculative decoding bridging the gap. Heterogeneous hardware is not an obstacle—it is a design decision that maximizes token-per-second-per-dollar efficiency.

References

[1] C. Jimenez, J. Yang, A. Wettig *et al.*, "Swe-bench: Can language models resolve real-world github issues?" in *ICLR 2024*, 2024.

[2] P. Gauthier, "Aider: Ai pair programming in the terminal," GitHub repository, 2024, https://github.com/Aider-AI/aider.

[3] S. Son, S. Yoon, J. Tang, S. Wang, L. Wolf, and I. Bogunovic, "Swe-router: Routing in multi-turn agentic software engineering tasks," in *ICML 2026 Workshop on Deep Learning for Code*, 2026. [Online]. Available: https://arxiv.org/abs/2607.00053

[4] C. Ong and X. Chen, "Routellm: Learning to route llms with preference data," *arXiv preprint arXiv:2406.18665*, 2024.

[5] L. Chen, M. Zaharia, and J. Zou, "Frugalgpt: How to use large language models while reducing cost and improving performance," in *ICML 2023*, 2023.

[6] J. Dong, J. Li, T. Zheng, and W. Lin, "Hybridflow: Resource-adaptive subtask routing for efficient edge-cloud llm inference," *arXiv preprint arXiv:2512.22137*, 2025.

[7] E. Labs, "Exo: Distributed inference on heterogeneous devices," Blog post / GitHub, 2025, https://blog.exolabs.net/nvidia-dgx-spark/.

[8] D. Ge, J. Gao, Q. Jiang, Y. Feng, and W. Ji, "Automatic task detection and heterogeneous llm speculative decoding," *arXiv preprint arXiv:2505.08600*, 2025.

[9] Z. Zhang, Z. Jiang, C. Jiang, M. Yu, S. Zheng, and H. Lin, "Swiftspec: Ultra-low latency llm decoding by scaling asynchronous speculative decoding," *arXiv preprint arXiv:2506.11309*, 2025.

[10] L. Zhang, Z. Zhang, Xubaizhou, R. Li, Z. Tian, and S. Mei, "Dovetail: A cpu/gpu heterogeneous speculative decoding for llm inference," in *EMNLP 2025*, 2025.

[11] J. Yang, C. Jimenez, A. Wettig *et al.*, "Swe-agent: Agent-computer interfaces enable automated software engineering," *arXiv preprint arXiv:2405.15793*, 2024.

[12] T. Xie, F. Zhou *et al.*, "Openhands: An open platform for ai software developers as generalist agents," *arXiv preprint arXiv:2407.16741*, 2024.

[13] S. Yao, J. Zhao, D. Yu *et al.*, "React: Synergizing reasoning and acting in language models," in *ICLR 2023*, 2023.

[14] S. Asthana, B. Zhang, C. DeLuca, H. Patel, and R. Mahindru, "Runtime-structured task decomposition for agentic coding systems," *arXiv preprint arXiv:2605.15425*, 2026.

[15] E. Yan, Y. He, S. Kim *et al.*, "Decoding speculative decoding," in *NAACL 2025*, 2025.

[16] Anonymous, "Hardware disaggregation with dgx spark: Optimizing llm tasks," Technical blog post, 2026, https://dasroot.net/posts/2026/05/hardware-disaggregation-dgx-spark-llm-inference/.

[17] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *ICML 2023*, 2023.

[18] Z. Zhang, X. Li *et al.*, "Mooncake: A kvcache-centric disaggregated architecture for llm serving," *arXiv preprint arXiv:2506.01234*, 2025.

[19] Y. Zhong, Y. Liang *et al.*, "Distserve: Disaggregating prefill and decoding for goodput-optimized llm serving," in *OSDI 2024*, 2024.

[20] Anonymous, "Harness engineering: Designing autonomous agent harnesses with locked metrics, durable logs, novelty gates, rollback, and human approval boundaries," *arXiv preprint arXiv:2607.XXXXX*, 2026.

[21] ——, "Codedelegator: Mitigating context pollution via role separation in code-as-action agents," *arXiv preprint arXiv:2601.14914*, 2026.

[22] ——, "Contextia: Deterministic context assembly for ai agents via explicit links," *arXiv preprint arXiv:2605.XXXXX*, 2026.

[23] ——, "Vessal: Frame-based architecture for kv-cache preservation in agent frameworks," *arXiv preprint arXiv:2606.XXXXX*, 2026.

[24] ——, "Latent briefing: Sharing orchestrator state with workers via task-guided kv cache compaction," *arXiv preprint arXiv:2605.XXXXX*, 2025.

[25] Th0rgal, "Dgx spark router: Automatic model switching on nvidia dgx spark," GitHub repository, 2026, https://github.com/Th0rgal/dgx-spark-router.

[26] A. Garg, S. Singha Roy, J. Jang, F. Brancasi, and S.-Y. Fu, "Hydra: Hybrid dynamic routing architecture for heterogeneous llm pools," *arXiv preprint arXiv:2605.17106*, 2026.

[27] Anonymous, "Routerwise: Joint resource allocation and routing for latency-aware multi-model llm serving," *arXiv preprint arXiv:2604.10907*, 2026.

[28] R. Zhu, Z. Jiang, C. Jin *et al.*, "Megascale-infer: Serving mixture-of-experts at scale with disaggregated expert parallelism," *arXiv preprint arXiv:2504.02263*, 2025.

[29] H. Wu, J. Zhu, Y. Li, H. Wang, B. Hou, and J. Zhai, "Specrouter: Adaptive routing for multi-level speculative decoding in large language models," *arXiv preprint arXiv:2505.07680*, 2025.

[30] Anonymous, "Aorchestra: Automating sub-agent creation for agentic orchestration," *arXiv preprint arXiv:2602.03786*, 2026.